Classes & Objects

Class

A **class** is a blueprint/template that defines:

- what data (**attributes**) objects of that type will have, and
- what behaviors (**methods**) they can perform.

It represents a general concept, **not** one specific item.

Syntax	Example

`class ClassName:`
`  # attributes + methods`
`  pass`
	`class Student:`
	`  Pass`
	Right now, Student is an empty blueprint.

Instance Attributes & __init__ (constructor)

We usually define how objects are initialized using **__init__**.

Syntax

```
class ClassName:
    def __init__(self, name, roll_no, grade):
        # instance attributes
        self.name = name
        self.roll_no = roll_no
        self.grade = grade
```

Key points:

- **__init__** runs automatically when you create an object.
- **self** refers to the specific object being created/used.
- **Attributes** attached to **self (like self.name)** are instance attributes.

Why __init__ and self?

- **To give every object proper initial data**

When you create a Student, you don't want an empty, meaningless object. You want: *"this student's name, roll number, grade"* set immediately.

Syntax

```
class ClassName:
    def __init__(self, name, roll_no, grade):
        # instance attributes
        self.name = name
        self.roll_no = roll_no
        self.grade = grade
```

- **To avoid repeating setup code**

Without __init__, you'd have to do:

```
s = Student()
s.name = "Ayaan"
s.roll_no = 5
s.grade = "A"
```

for every object. Boring, error-prone, easy to forget something.

- **To enforce required information**

By writing __init__(self, name, roll_no, grade), you're saying:
"You must give me these three things if you want a valid Student."

- **To tie data to that specific object**

Using self.something = ... stores the values inside that particular object, not globally.

Class vs Instance Level

Syntax

```
class Student:
    school_name = "ABC Public School"  # class attribute (shared)

    def __init__(self, name):
        self.name = name               # instance attribute (per object)
```

Class attribute:
- belongs to the class,
- shared by all objects (unless overridden per object).

Instance attribute:
- unique to each object.

Methods (Behavior)

Syntax

```
class Student:
    def __init__(self, name, roll_no, grade):
        # instance attributes
        self.name = name
        self.roll_no = roll_no
        self.grade = grade

    # instance method
    def display_info(self):
        print(f"Name: {self.name}, Roll: {self.roll_no}, Grade: {self.grade}")

    def update_grade(self, new_grade):
        self.grade = new_grade
```

Key points:
First parameter of an instance method is conventionally named **self**.
Through **self**, the method can:

- **read/update** the object's **attributes**,
- **call** other **methods** of the same object.

Object

An **object** is a concrete **instance of a class**.

It is a real **“thing”** in memory created from the blueprint.

Each object has:

- its own **state** (values of attributes),
- access to its **behavior** (methods defined in its class),
- its own **identity** (it’s a distinct entity).

Syntax	Creating Objects (Instantiation)
`variable = Classname(...)`	`s1 = Student("Ayesha", 101, "A")` `s2 = Student("Rahul", 102, "B")`

Object - Example

Class

Student (name, roll, grade)

Object

Student 1

Name: Ayesha,
Roll: 101
Grade: “A”)

Student 2

Name: Rahul,
Roll: 102
Grade: “B”)

Object: Accessing Attributes & Methods

Syntax

```
print(s1.name)              # Accessing attribute
s1.display_info()           # Calling method on object
s2.update_grade("A+")
s2.display_info()
```

Key points:
We use dot **(.)** notation:

- **object.attribute** `# Note: Attribute do not accept ()`
- **object.method()**

Inside the class, we use **self** to mean **"this object"**.

Outside the class, we use the **variable name (s1, s2)** that refers to the **object**.

Object: Identity | State | Behaviour

```
s1 = Student("Ayesha", 101, "A")
```

Identity:
Which specific object in memory s1 refers to.

State:
`{"name": "Ayesha", "roll_no": 101, "grade": "A"}`.

Behavior:
Methods `display_info`, `update_grade` it can perform

Syntax

```
class Student:
    def __init__(self, name, roll_no, grade):
        # instance attributes
        self.name = name
        self.roll_no = roll_no
        self.grade = grade

    # instance method
    def display_info(self):
        pass

    def update_grade(self, new_grade):
        pass
```

Example

Syntax

```
class BankAccount:
    # class attribute (optional example)
    bank_name = "ABC Bank"

    def __init__(self, owner, balance=0):
        # instance attributes
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
        else:
            print("Invalid withdrawal")

    def show_balance(self):
        print(f"{self.owner}'s balance:
{self.balance}")
```

Syntax

```
# creating objects
acc1 = BankAccount("Neha", 1000)
acc2 = BankAccount("Aman", 500)

# using objects
acc1.deposit(200)
acc2.withdraw(100)

acc1.show_balance()    # Neha's balance: 1200
acc2.show_balance()    # Aman's balance: 400
```

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Class Attributes
Class Methods